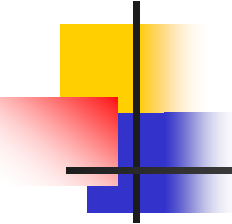


算法设计与分析

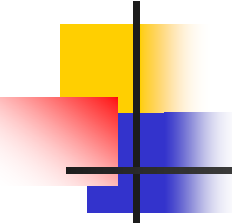
Analysis and Design of Algorithm

Lesson 14



关于上机

- **时间：**第9-16周每周五2-5节
- **地点：**教1-504（自带电脑）
- **平台：**洛谷 <https://www.luogu.com.cn>
- **要求：**
 1. 每次上机采取班级内部比赛的形式（题目在洛谷平台发布）。
 2. 在线提交代码并通过后，需要让助教查看（随机抽一部分同学答辩）后方可离开。
 3. 若课上未及时完成或提交未Accept，课后仍可提交，但会酌情减分。
 4. 每次课都需要提供实验报告，模版后续会提供。
 5. 最终成绩结合“代码通过测试点的数量”、“代码时空复杂度”、“实验报告质量”、“代码查重”等多方面因素综合评定。



要点回顾

- 分支限界：一种与回溯法类似的算法
 - 将求解问题建模为搜索解空间树
 - 通常用限界函数估算每个分支最优值
 - 优先选择当前看来最好的分支
 - 搜索策略一般采用宽度优先搜索
 - 搜索过程中剪枝（2个条件）
 - 不满足约束条件
 - 限界函数值不优于当前的界
- 分支限界实例
 - 背包问题

分支限界解组合优化问题

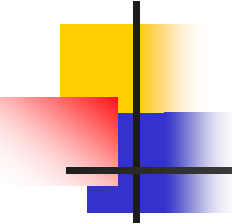
■ 背包问题

背包限重为10

物品 <i>i</i> 属性	1	2	3	4
价值 v_i	1	3	5	9
重量 w_i	2	3	4	7

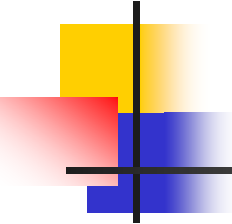
$$\max (x_1 + 3x_2 + 5x_3 + 9x_4)$$

$$s.t. \quad \begin{cases} 2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10 \\ x_i \in N, i = 1, 2, 3, 4 \end{cases}$$



限界函数的设定

- 对结点 $\langle x_1, x_2, \dots, x_k \rangle$, 估计以该结点为根的子树中可行解的上界
- 按单位重量的价值 v_i / w_i 从大到小排序
- 限界函数=已装入价值+ Δ
 - Δ : 还可以继续装入最大价值的上界
 - $\Delta = \text{背包剩余重量} \times v_{k+1} / w_{k+1}$ (可装)
 - $\Delta = 0$ (不可装)



实例：背包问题

$$\max (x_1 + 3x_2 + 5x_3 + 9x_4)$$

$$s.t. \quad \begin{cases} 2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10 \\ x_i \in N, i = 1, 2, 3, 4 \end{cases}$$

对**变元重新排序**使得 $\frac{v_i}{w_i} \geq \frac{v_{i+1}}{w_{i+1}}$

排序后

$$\max (9x_1 + 5x_2 + 3x_3 + x_4)$$

$$s.t. \quad \begin{cases} 7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10 \\ x_i \in N, i = 1, 2, 3, 4 \end{cases}$$

限界函数与分支策略

- 结点 $\langle x_1, x_2, \dots, x_k \rangle$ 的**限界函数** F

若对某个 $j > k$ 有 $b - \sum_{i=1}^k w_i x_i \geq w_j$

$$F = \sum_{i=1}^k v_i x_i + (b - \sum_{i=1}^k w_i x_i) \frac{v_{k+1}}{w_{k+1}}$$

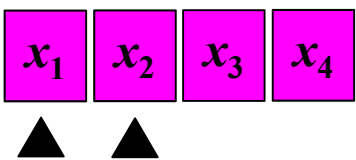
否则 $F = \sum_{i=1}^k v_i x_i$

分支策略——深度优先+限界函数优先

实例 $\max (9x_1 + 5x_2 + 3x_3 + x_4)$

$s.t. \begin{cases} 7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10 \\ x_i \in N, i=1,2,3,4 \end{cases}$

队列

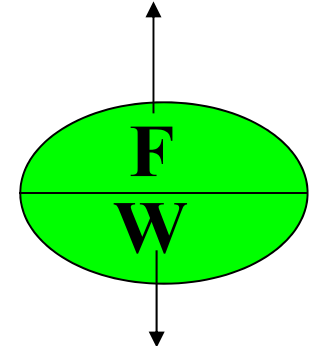


若对某个 $j > k$ 有 $b - \sum_{i=1}^k w_i x_i \geq w_j$

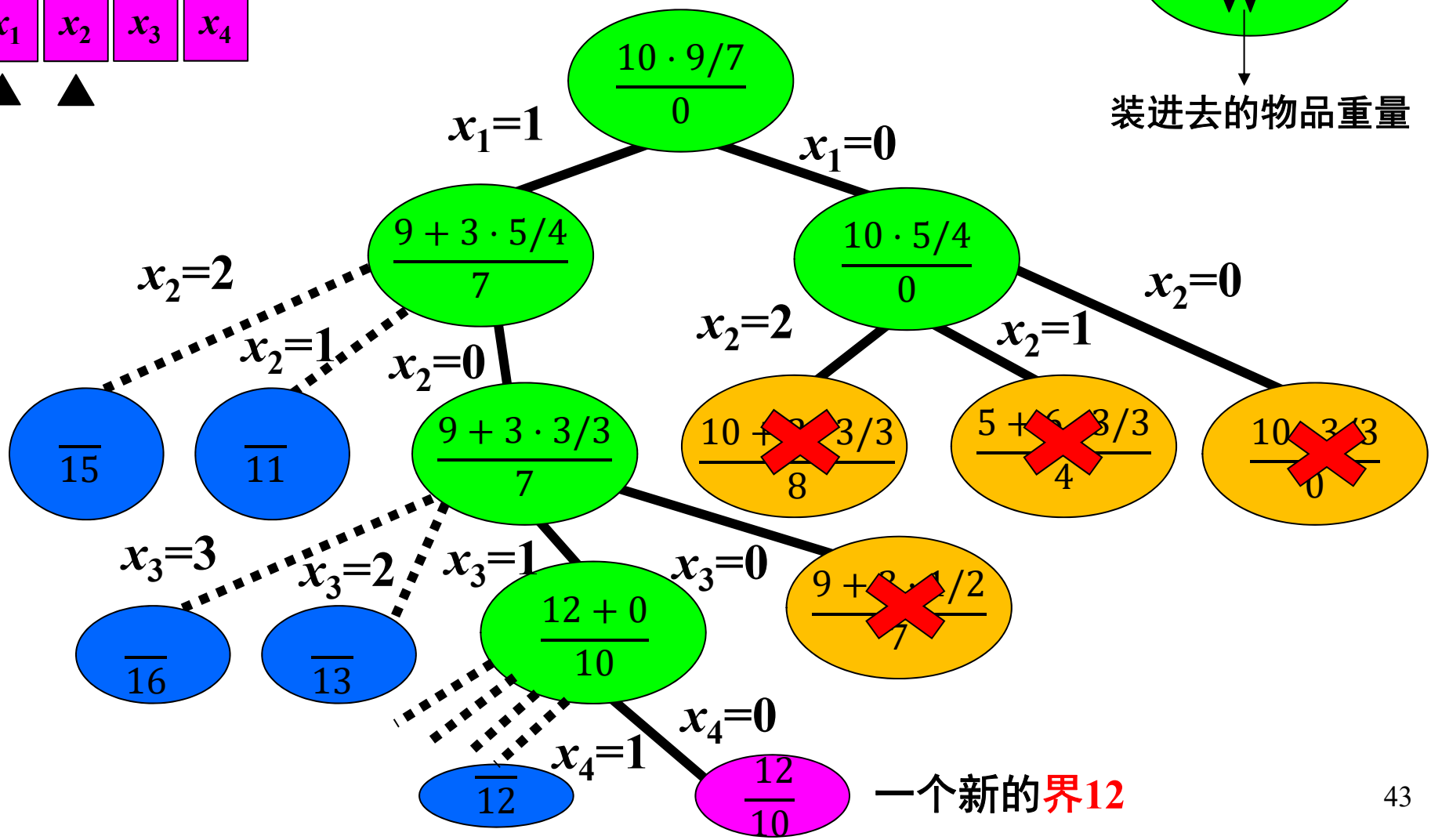
$F = \sum_{i=1}^k v_i x_i + (b - \sum_{i=1}^k w_i x_i) \frac{v_{k+1}}{w_{k+1}}$

否则 $F = \sum_{i=1}^k v_i x_i$

限界函数计算的值



装进去的物品重量



0-1背包问题



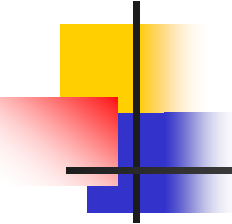
■ 实例

- 4种物品，重量 w_i 和价值 v_i 分别为
- $v_1 = 1, v_2 = 3, v_3 = 5, v_4 = 10$
- $w_1 = 2, w_2 = 3, w_3 = 6, w_4 = 7$
- 背包重量限制为10

■ 建模：

最大化 $x_1 + 3x_2 + 5x_3 + 10x_4$

满足约束条件
$$\begin{cases} 2x_1 + 3x_2 + 6x_3 + 7x_4 \leq 10 \\ x_i \in \{0,1\}, \quad i = 1, 2, 3, 4 \end{cases}$$



0-1背包问题—限界函数

- 按 v_i/w_i 从大到小排序, $i = 1, 2, \dots, n$
- 假设位于结点 $\langle x_1, x_2, \dots, x_k \rangle$
- 限界函数=已装入价值+ Δ
 - Δ : 还可继续装入最大价值的上界
 - Δ =背包剩余重量 $\times v_{k+1}/w_{k+1}$ (可装)
 - $\Delta=0$ (不可装)

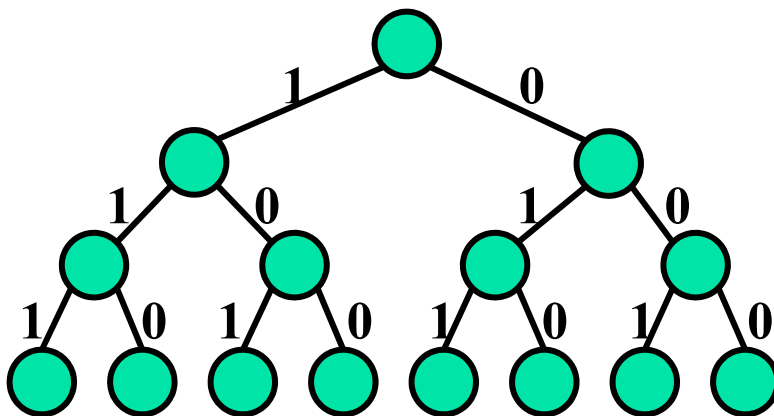
0-1背包问题—分支限界法

■ 基本思想

1. 将物品按 v_i/w_i 从大到小排序，确定解空间树
2. 从空集 $\emptyset$ 和仅含空集 $\emptyset$ 的优先队列开始
3. 选择计算节点队列中代价值最高的节点并扩展
4. 若扩展出节点不被剪枝，将节点插入节点队列
5. 反复2~3步，直到优先队列为空时为止

■ 限界函数

- 已装入价值 $+\Delta$



0-1背包问题—分支限界法

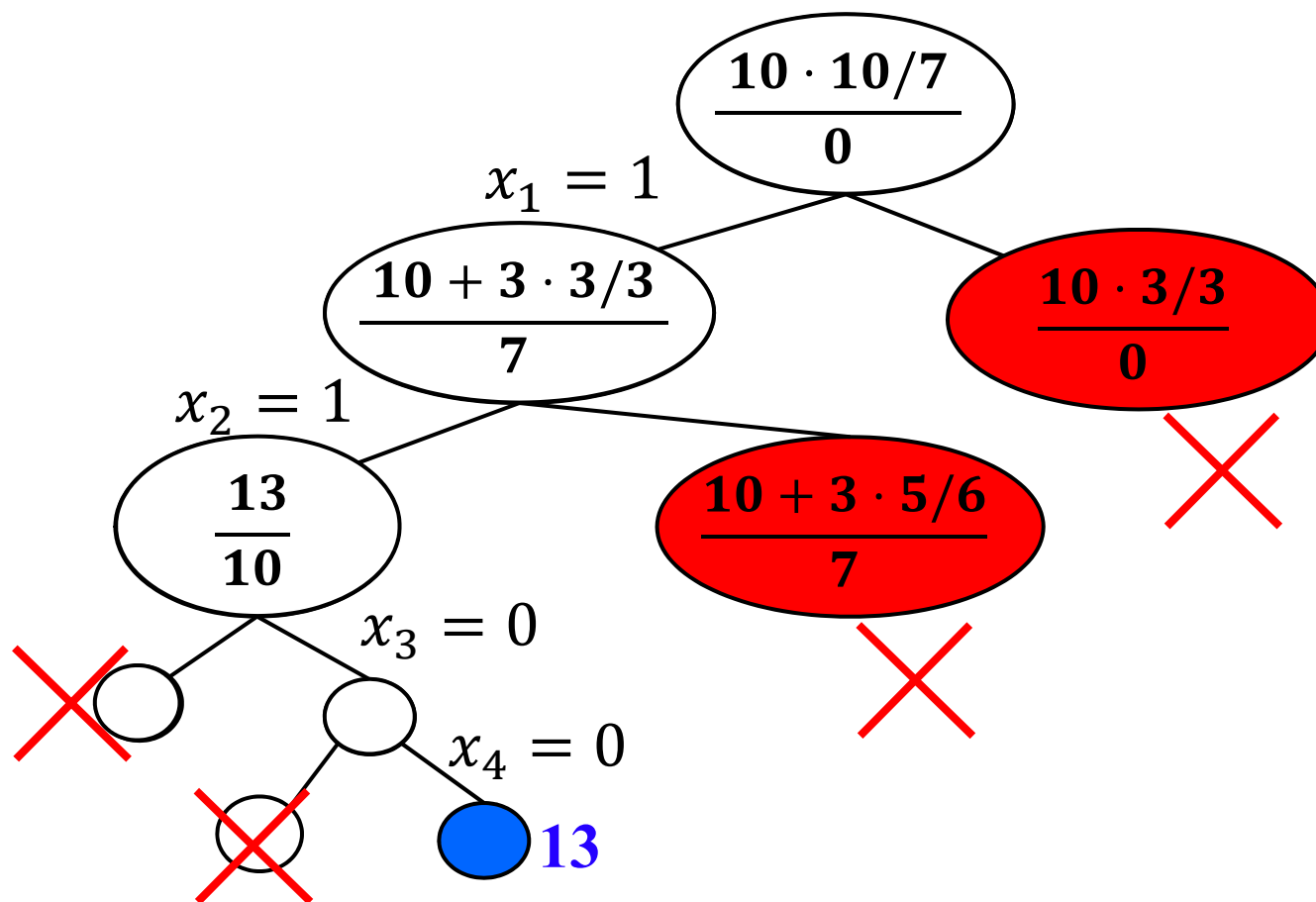
最大化 $10x_1 + 3x_2 + 5x_3 + x_4$

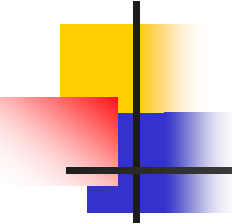
满足 $7x_1 + 3x_2 + 6x_3 + 2x_4 \leq 10; x_i \in \{0,1\}, i = 1, 2, 3, 4$

限界函数计算的值

$$\frac{F}{W}$$

装进去会不会超重



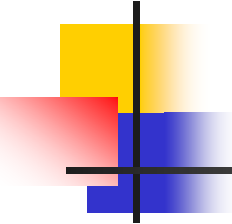


TSP问题

输入： 城市集 $C=\{c_1, c_2, \dots, c_n\}$, 距离 $d(c_i, c_j)=d(c_j, c_i)$

解： $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$



算法设计

- **解向量**为: $\langle 1, i_1, i_2, \dots, i_{n-1} \rangle$, 其中 $i_1, i_2, \dots, i_{n-1}$ 为 $\{2, 3, \dots, n\}$ 的排列
- 搜索空间为**排列树**, 结点 $\langle i_1, i_2, \dots, i_k \rangle$ 表示得到 k 步路线
- **约束条件**: 令 $B = \{i_1, i_2, \dots, i_k\}$ 则 $i_{k+1} \in \{2, 3, \dots, n\} - B$, 即每个结点只能访问一次

限界函数与界

- **界**：当前得到的最短巡回路线长度
- **限界函数**：设顶点 c_i 出发的最短边长度为 l_i ， d_j 为选定巡回路线中第 j 段的长度

$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

已走过
路径长

剩余长
度下界

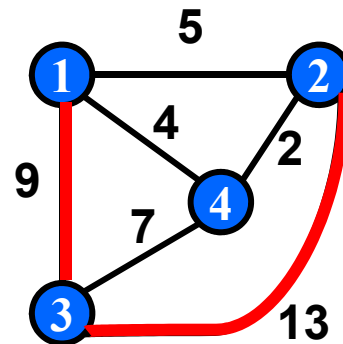
限界函数

$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

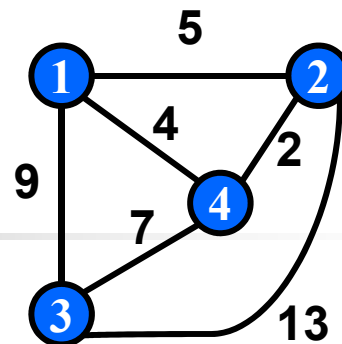
部分路线<1, 3, 2>

$$L=9+13+2+2=26$$

- 9+13为走过的路径长度
- 后两项分别为从结点2及结点4出发的最短边长

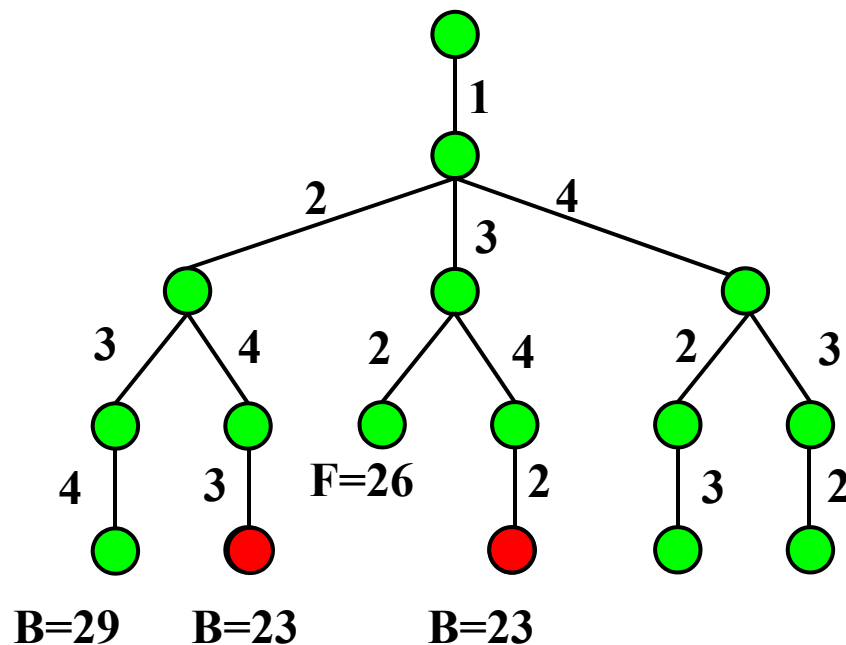


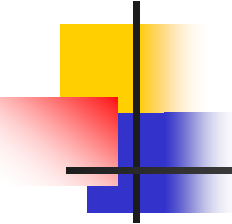
搜索树



深度优先遍历搜索树

- 第1个界: $\langle 1, 2, 3, 4 \rangle$, $B=29$
- 第2个界: $\langle 1, 2, 4, 3 \rangle$, $B=23$
- 结点 $\langle 1, 3, 2 \rangle$: 限界函数值 $26 > 23$, 不再搜索, 返回 $\langle 1, 3 \rangle$, 右子树向下
- 结点 $\langle 1, 3, 4 \rangle$: 限界函数值 $9 + 7 + 2 + 2 = 20 < 23$, 继续, 得到可行解 $\langle 1, 3, 4, 2 \rangle$, 长度23
- 回溯到结点 $\langle 1 \rangle$, 沿 $\langle 1, 4 \rangle$ 向下
- ... **→ 最优解:** $\langle 1, 2, 4, 3 \rangle$ 或 $\langle 1, 3, 4, 2 \rangle$, 长度23





算法分析

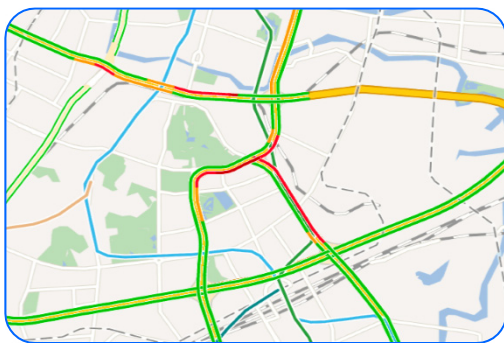
- 搜索树的树叶个数： $O((n-1)!)$ ，每片树叶对应1条路径，每条路径有 n 个结点
- 每个结点限界函数计算时间 $O(1)$ ，每条路径的计算时间 $O(n)$
- 最坏情况下算法的时间 $O(n!)$
 - 但是实际运行起来，效果会比最坏的好。

非对称TSP问题

■ 问题定义

- 城市集合: $C = \{c_1, c_2, \dots, c_n\}$
- 城市距离: $d(c_i, c_j)$
- 距离不对称: $d(c_i, c_j) \neq d(c_j, c_i)$

■ 目标: 求遍历所有城市（不重复）的最短路径



道路拥堵情况下
送快递问题



考虑城市单行线
送货问题



全国巡回募捐
路线安排问题

守望相助、武汉加油巡回募捐活动

- 庚子年爱心募捐活动：
 - **地点：**武汉、北京、上海、广州、深圳、南京、杭州、成都、重庆、厦门
 - **线路：**从武汉出发，跑遍各大城市，回到武汉
- **目标：**考虑机票价格，确定旅费最少的线路

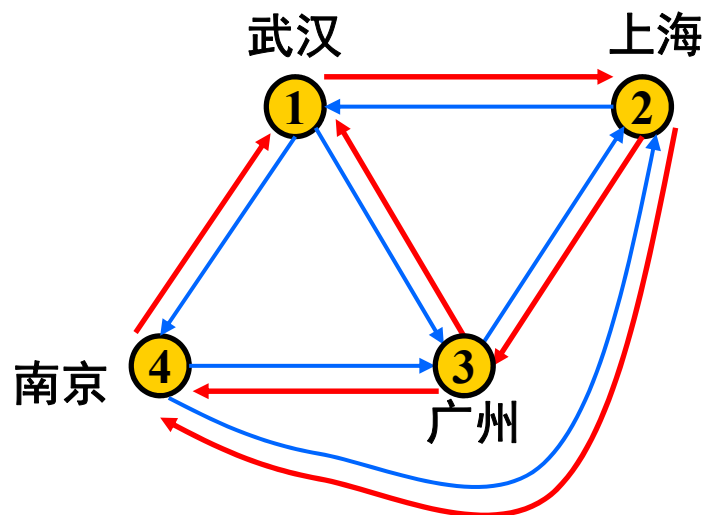


运费	武汉	北京	上海	
武汉	0	500	600	
北京	100	0	800	
上海	1000	200	0	
.....				

非对称旅行商问题的解空间

■ 实例

	武汉	上海	广州	南京
武汉	0	500	600	100
上海	100	0	800	500
广州	1000	200	0	2000
南京	400	400	100	0



■ 最优解

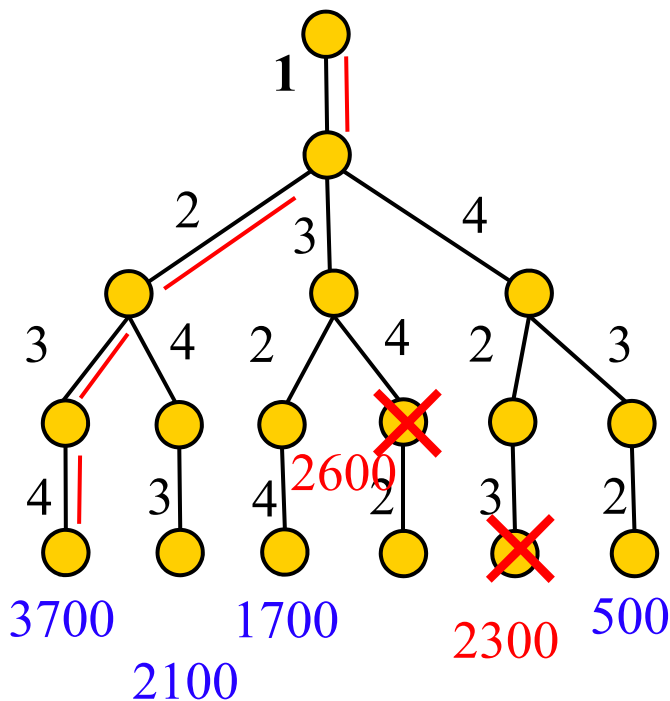
- 解的表示: $\langle 1, 4, 3, 2 \rangle$
- 路线: 武汉 $\rightarrow$ 南京 $\rightarrow$ 广州 $\rightarrow$ 上海 $\rightarrow$ 武汉
- 总运费: $100 + 100 + 200 + 100 = 500$

非对称旅行商问题的解空间树

	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

回溯法

- 深度优先遍历解空间树
- 剪枝函数：比较当前解与当前最优解



非对称旅行商问题的解

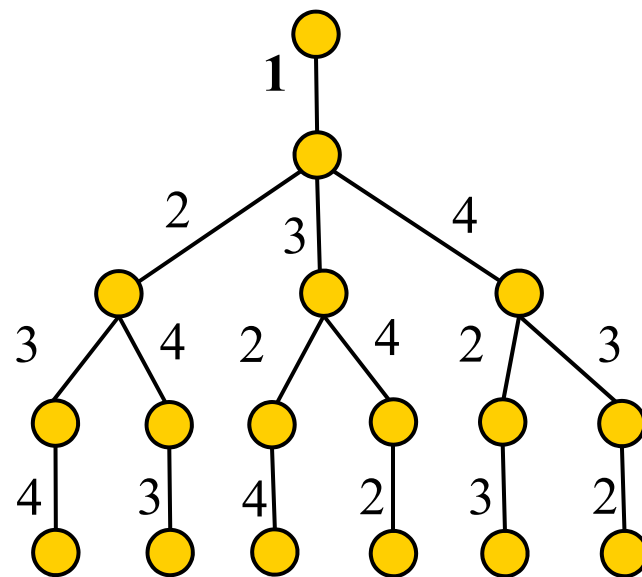
	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

■ 如何尽快寻找最优解

- 不用深度优先搜索
- 优先访问更靠近最优解的节点
- 设置**限界函数**计算优先级
- 利用**优先级队列**管理节点

■ 如何设计限界函数

- **当前解的值** + **未来的最优解估计值**



非对称旅行商问题的解

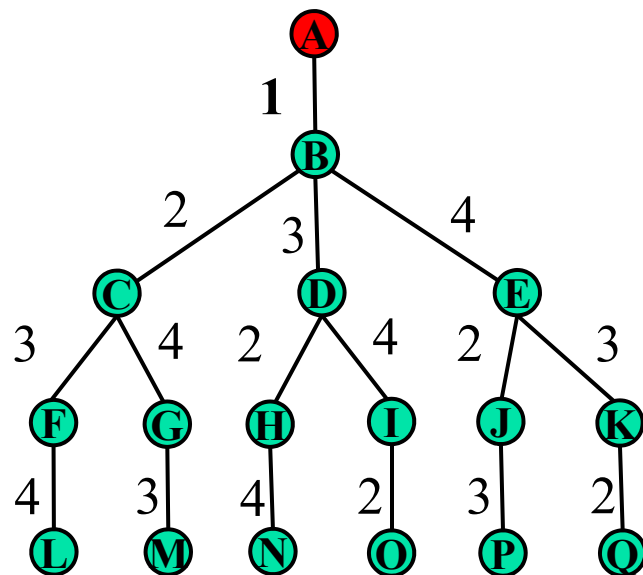
	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

节点颜色

红色：优先级队列中的节点
绿色：未被访问的节点
白色：已经完成访问的节点

未来最优解估计值=

每一个未选城市最低出发运费之和 +
当前选中城市的最低出发运费



节点	当前值	未来最优值	总代价
A	0	500	500

查表

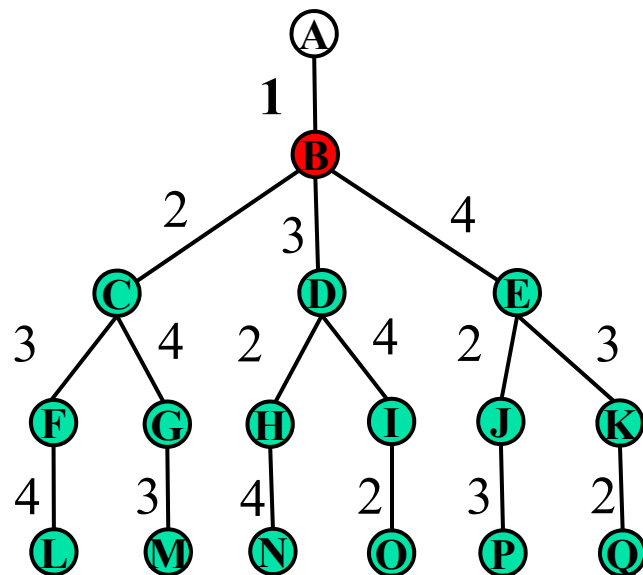
(100+200+100+100) + 0

非对称旅行商问题的解

	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

节点颜色

红色：优先级队列中的节点
绿色：未被访问的节点
白色：已经完成访问的节点



节点	当前值	未来最优值	总代价
B	0	500	500

未来最优解估计值=

每一个未选城市最低出发运费之和 +
当前选中城市的最低出发运费

查表

(100+200+100) + 100

非对称旅行商问题的解

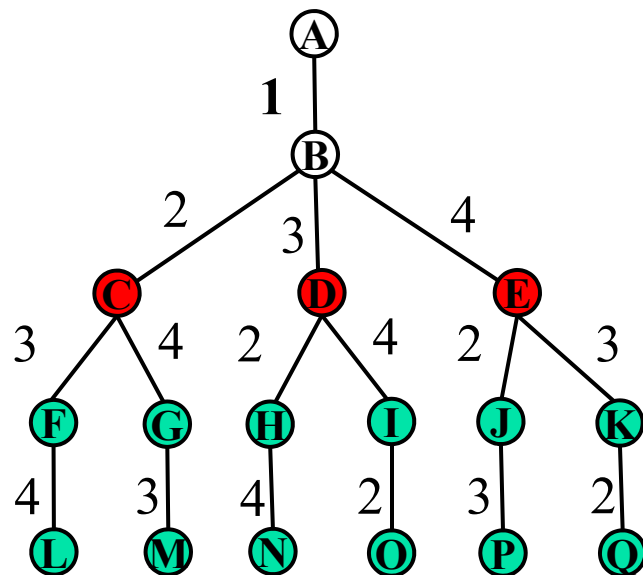
	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

节点颜色

红色：优先级队列中的节点
绿色：未被访问的节点
白色：已经完成访问的节点

未来最优解估计值=

每一个未选城市最低出发运费之和 +
当前选中城市的最低出发运费



节点	当前值	未来最优值	总代价
C			
D			
E			

查表

非对称旅行商问题的解

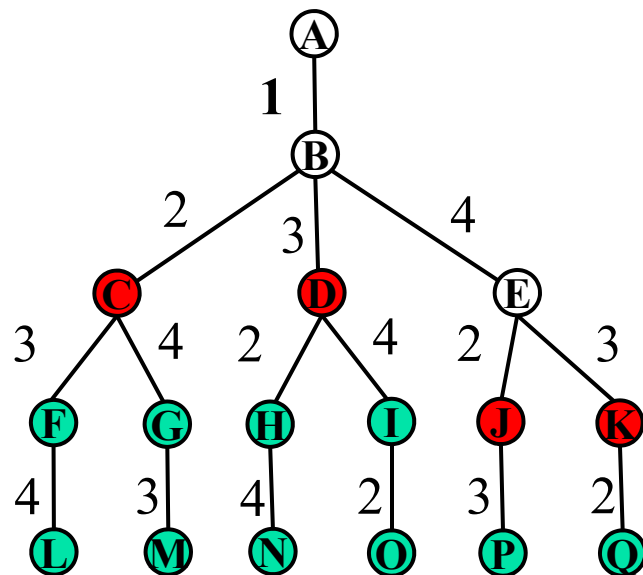
	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

节点颜色

红色：优先级队列中的节点
绿色：未被访问的节点
白色：已经完成访问的节点

未来最优解估计值=

每一个未选城市最低出发运费之和 +
当前选中城市的最低出发运费



节点	当前值	未来最优值	总代价
C	500	400	900
D	600	400	1000
J			
K			

非对称旅行商问题的解

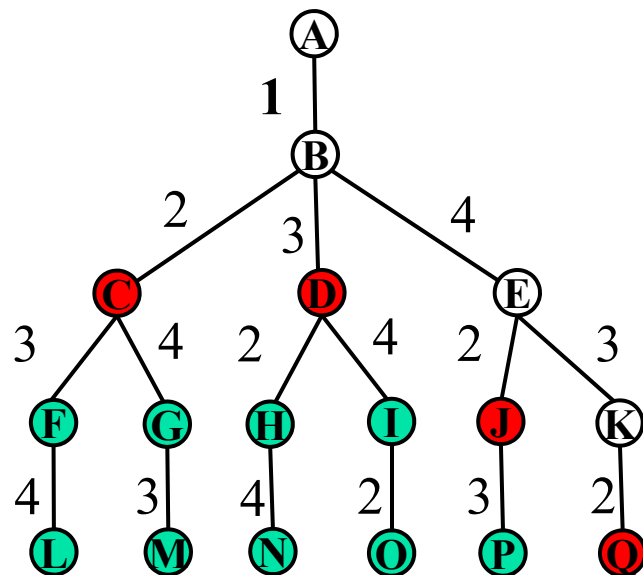
	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

节点颜色

红色：优先级队列中的节点
绿色：未被访问的节点
白色：已经完成访问的节点

未来最优解估计值=

每一个未选城市最低出发运费之和 +
当前选中城市的最低出发运费



节点	当前值	未来最优值	总代价
C	500	400	900
D	600	400	1000
J	500	300	800
Q	400	100	500

当前
最优

非对称旅行商问题的解

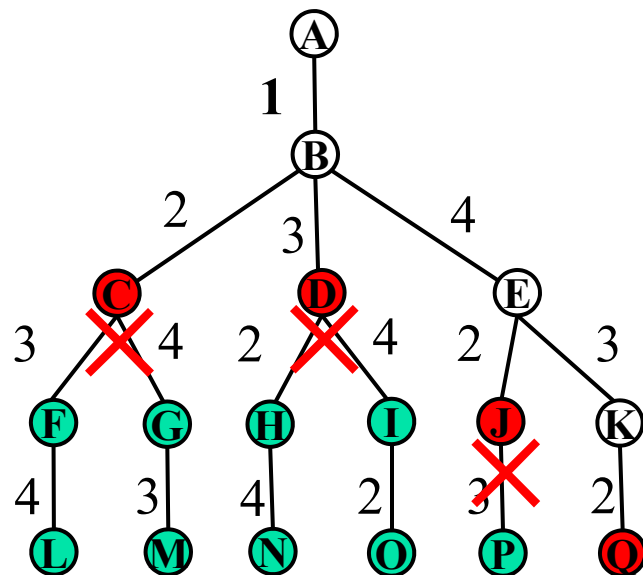
	1	2	3	4
1	0	500	600	100
2	100	0	800	500
3	1000	200	0	2000
4	400	400	100	0

节点颜色

红色：优先级队列中的节点
绿色：未被访问的节点
白色：已经完成访问的节点

未来最优解估计值=

每一个未选城市最低出发运费之和 +
当前选中城市的最低出发运费



节点	当前值	未来最优值	总代价
C	500	400	900
D	600	400	1000
J	500	300	800
Q	400	100	500

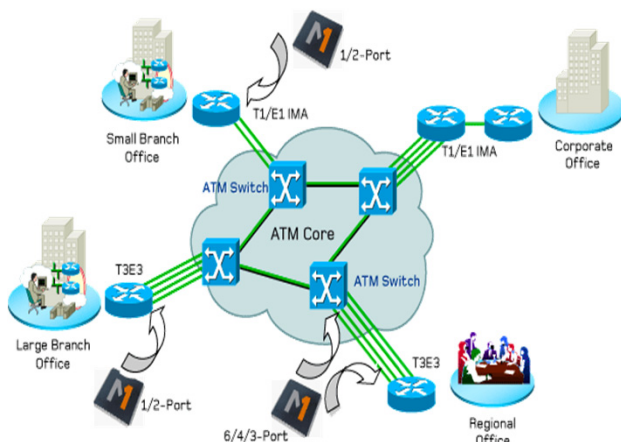
当前
最优

最短路径问题（回顾）

- 最短路径问题是图论研究中的一个经典问题，旨在寻找图（由结点和路径组成的）中两结点之间的最短路径。



地图导航



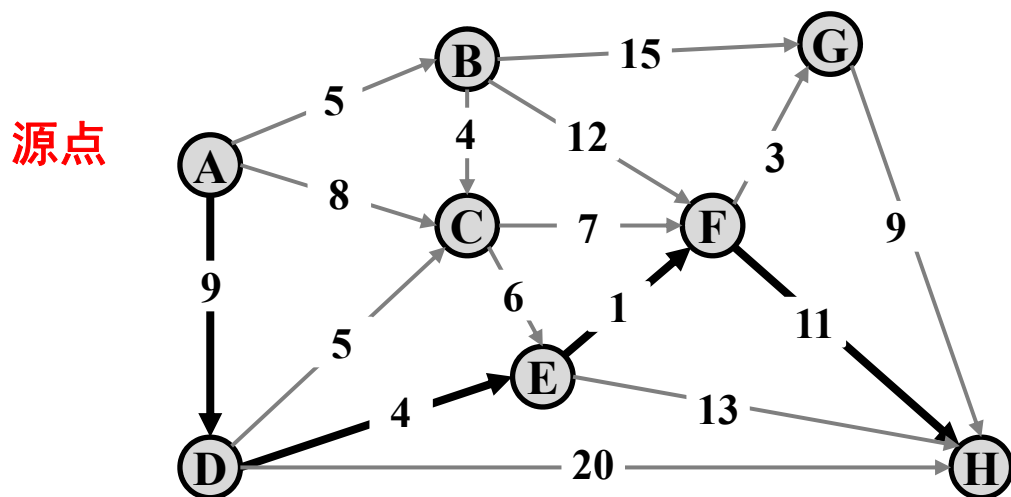
网络路由



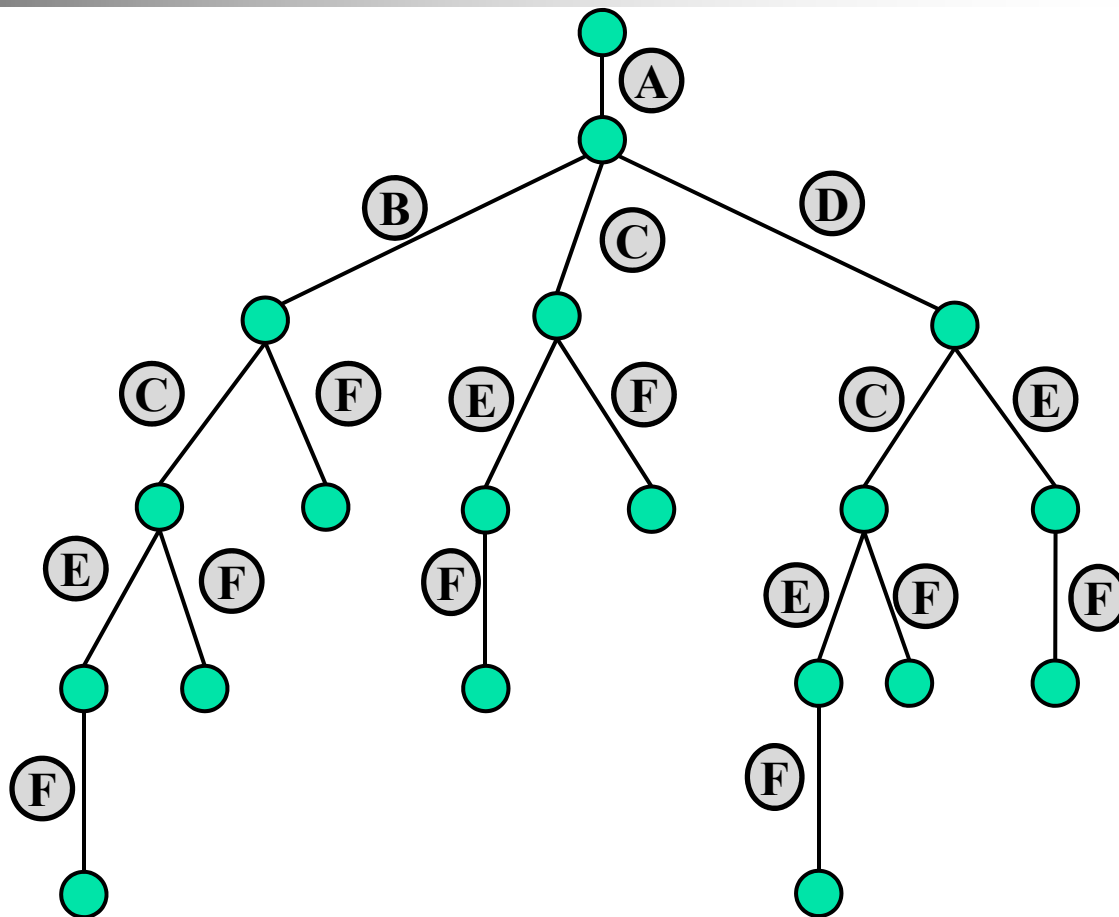
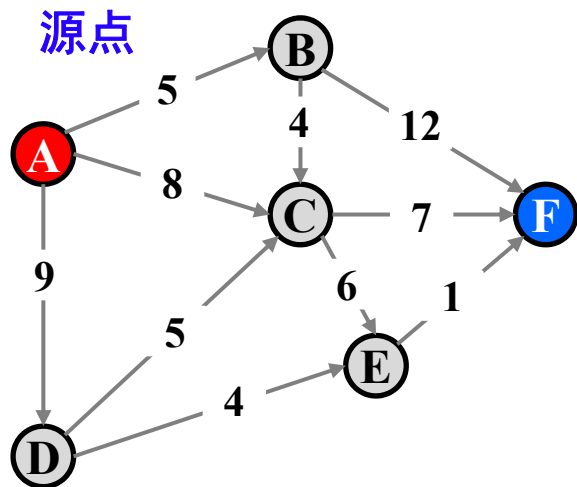
极速外卖

单源最短路径（回顾）

- 给定带权有向图 $G=(V, E)$ ，其中每条边的权是非负实数。给定 V 中的一个顶点作为源点，求源点到所有其他各顶点的最短路长度。

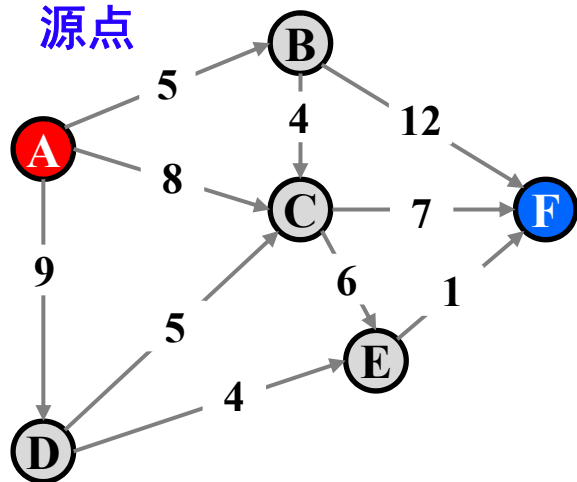


计算距离(A,F)的解空间树



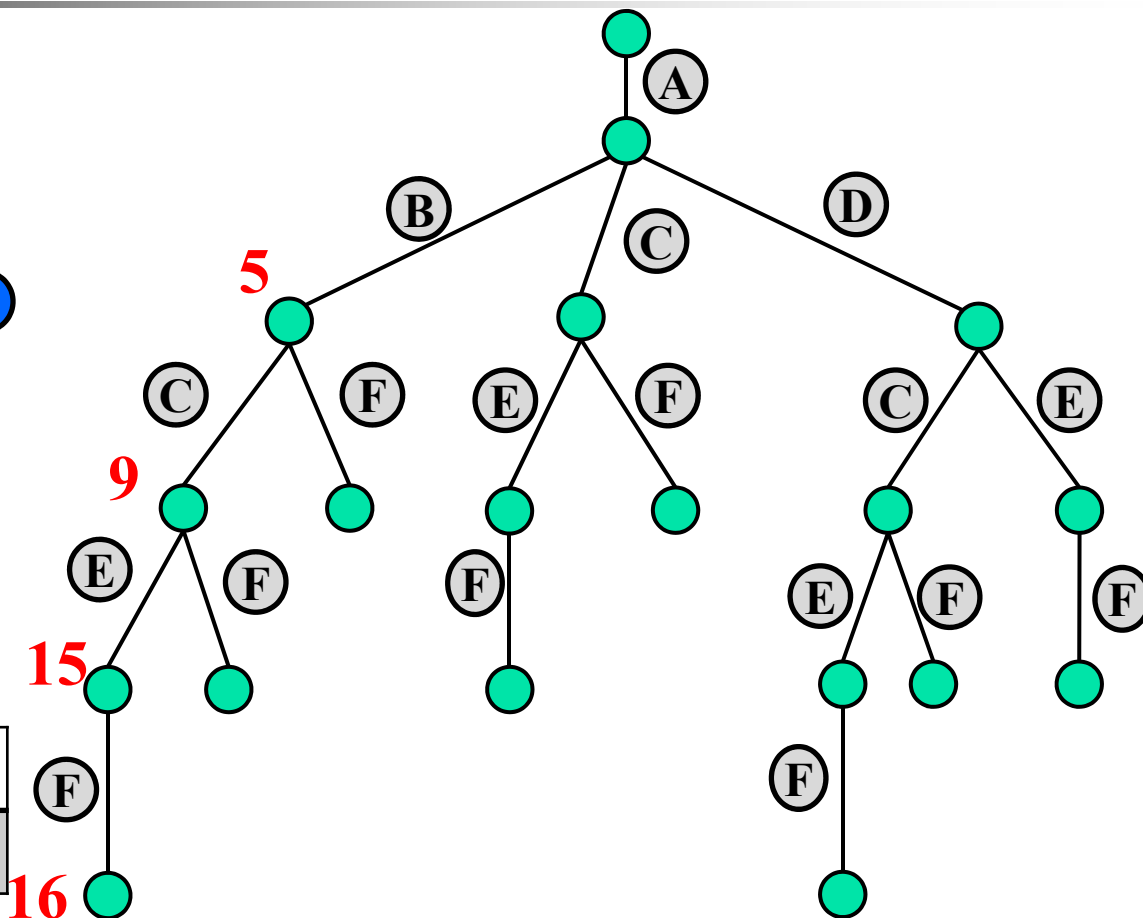
计算距离(A,F)——回溯法

源点



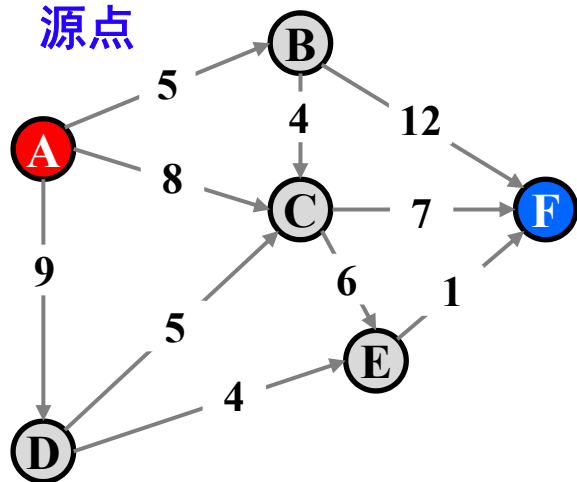
当前最短距离

	B	C	D	E	F
dist	5	9		15	16



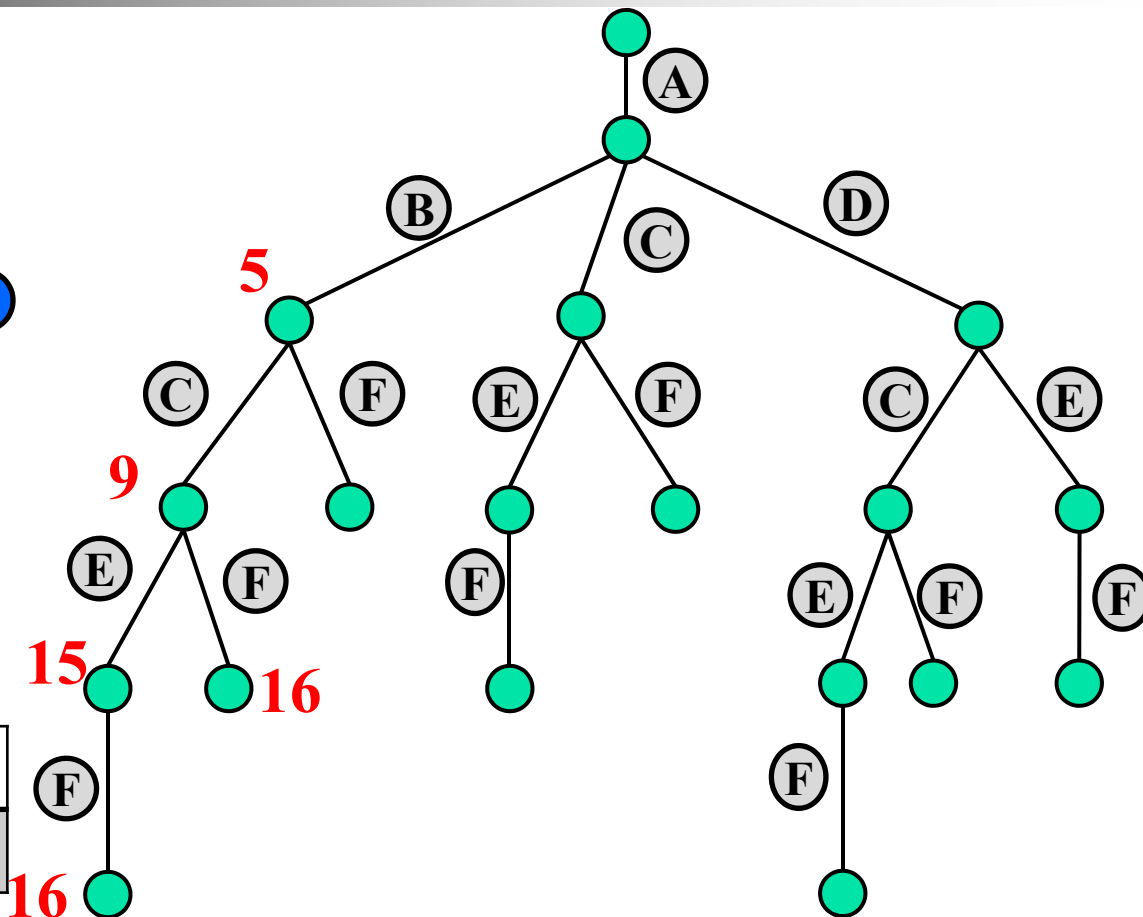
计算距离(A,F)—回溯法

源点



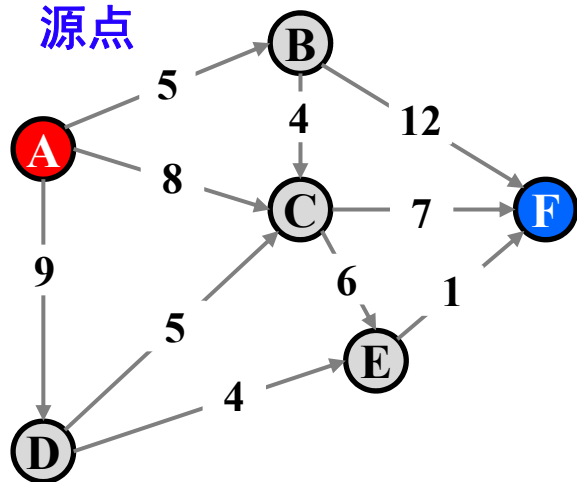
当前最短距离

	B	C	D	E	F
dist	5	9		15	16



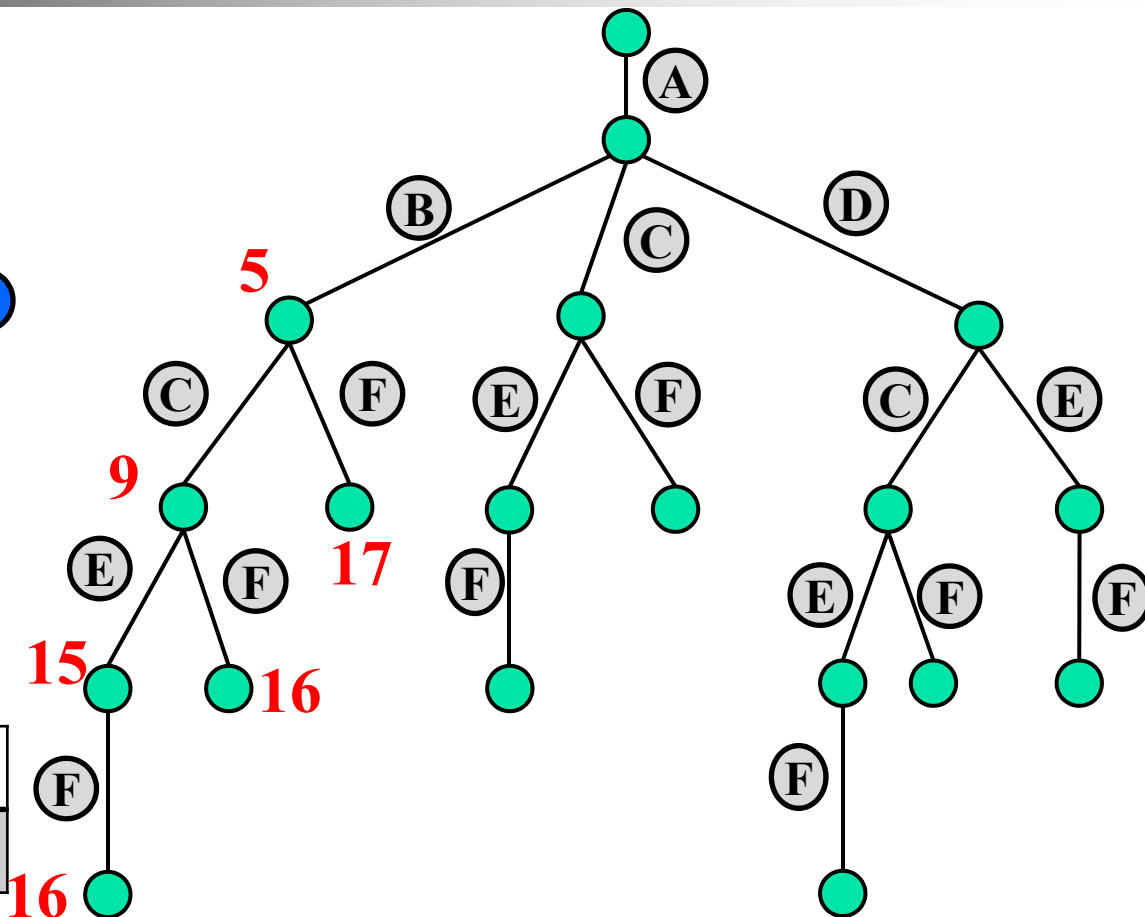
计算距离(A,F)——回溯法

源点



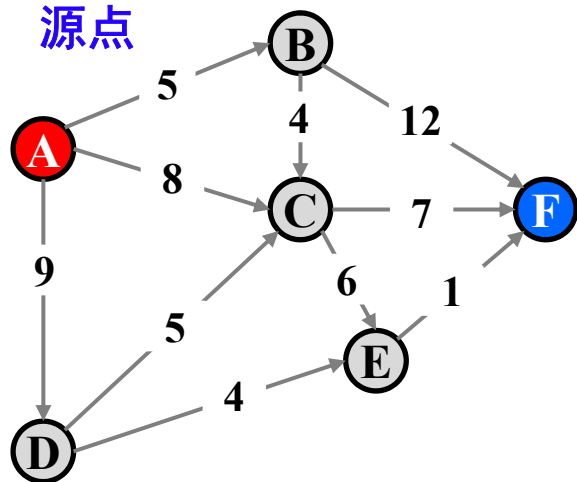
当前最短距离

	B	C	D	E	F
dist	5	9		15	16



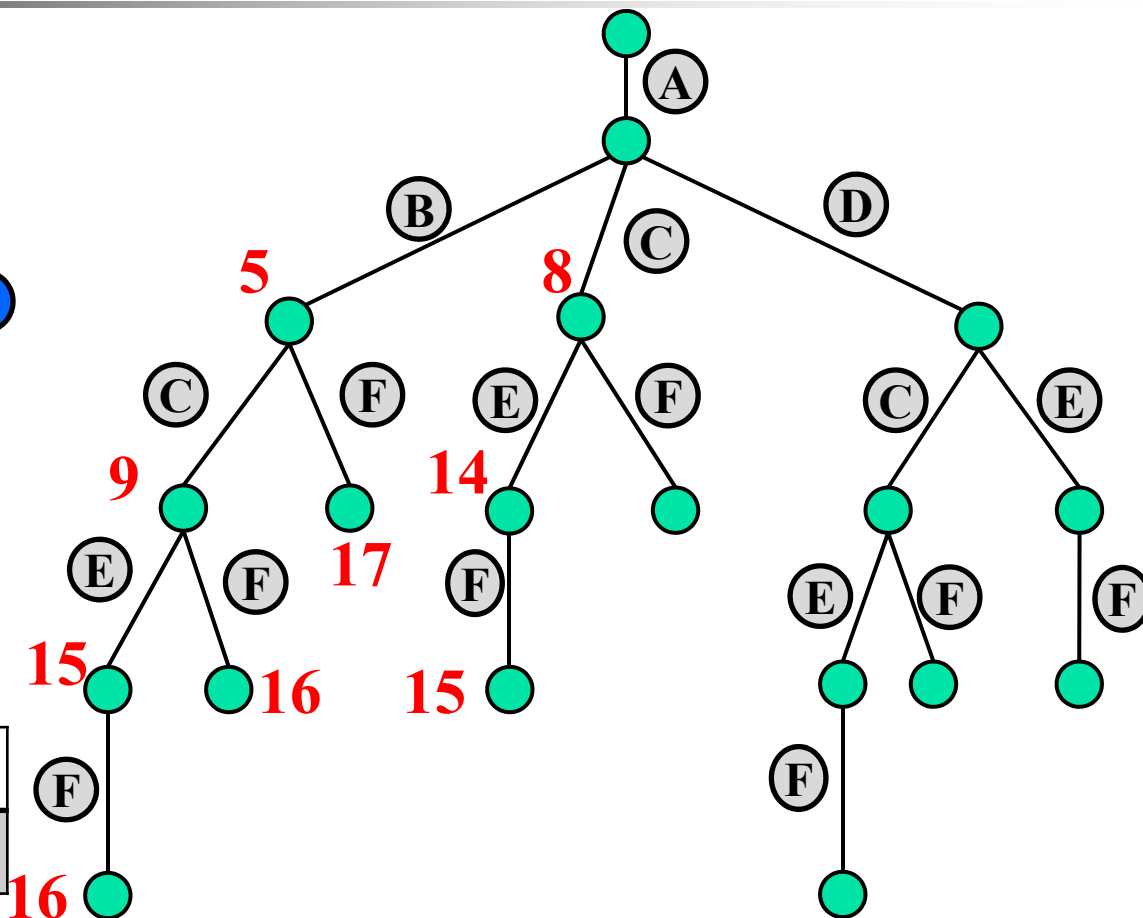
计算距离(A,F)——回溯法

源点



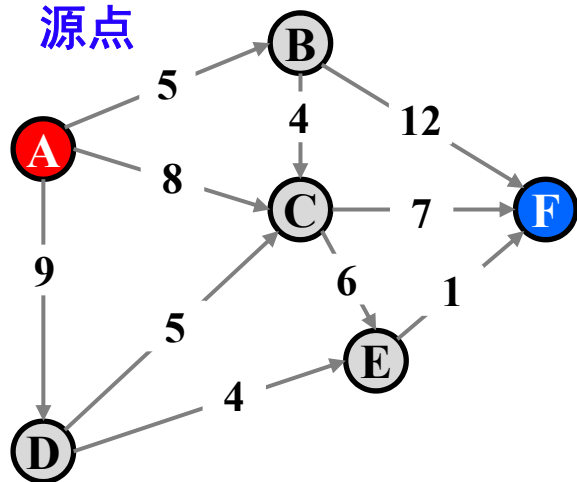
当前最短距离

	B	C	D	E	F
dist	5	8		14	15



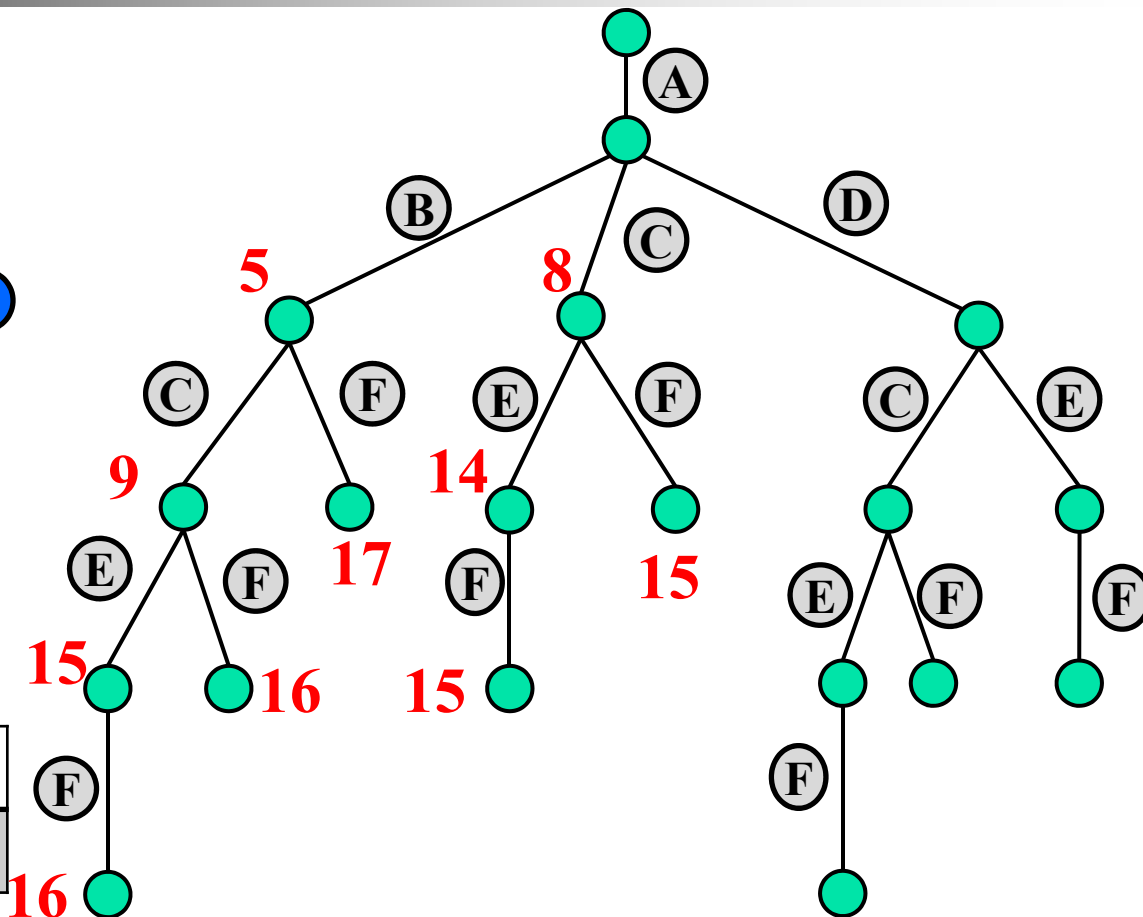
计算距离(A,F)——回溯法

源点

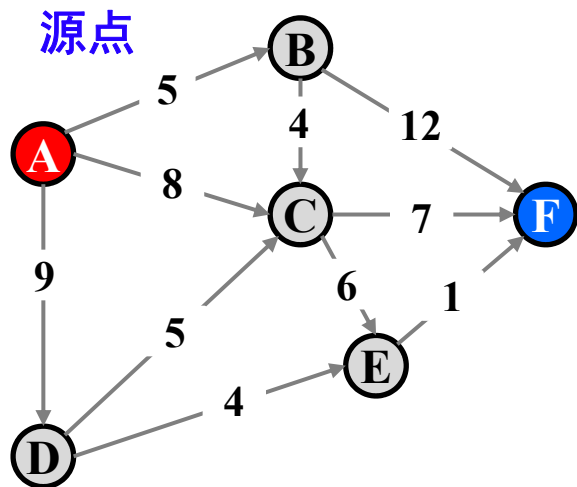


当前最短距离

	B	C	D	E	F
dist	5	8		14	15

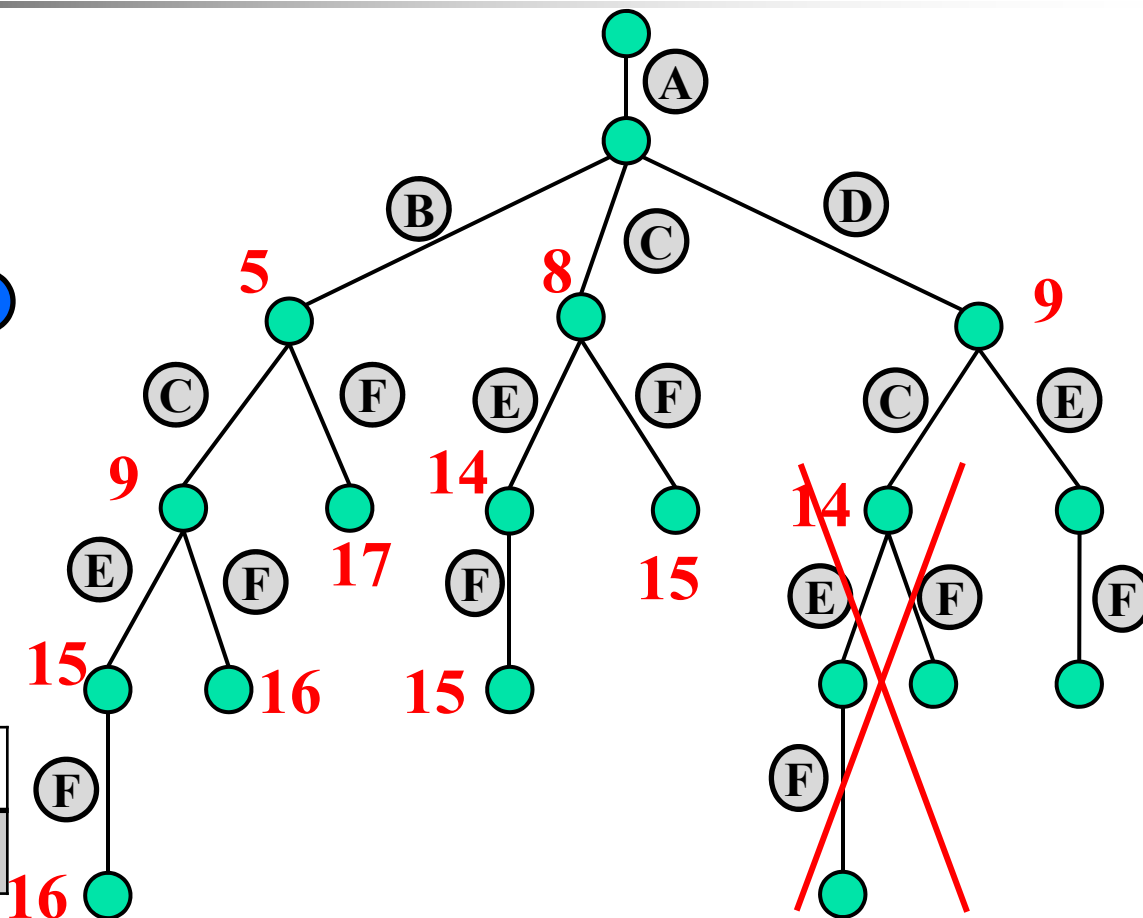


计算距离(A,F)——回溯法



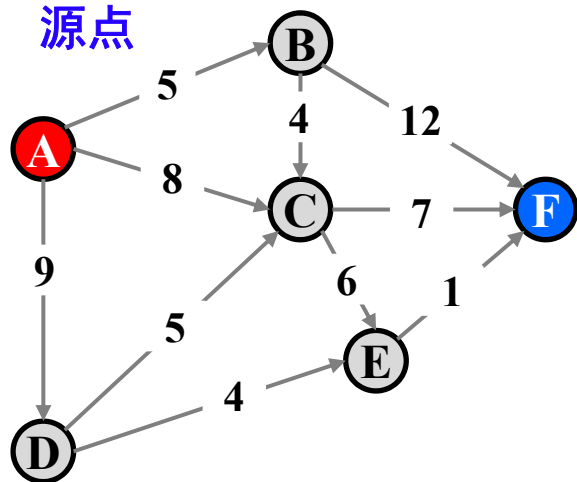
当前最短距离

	B	C	D	E	F
dist	5	8	9	14	15



计算距离(A,F)——回溯法

源点



当前最短距离

	B	C	D	E	F
dist	5	8	9	13	14

